

**SSC0600 – Introdução à Ciência de Computação I – 2026 (ICC1 - Teoria)****Professor responsável:** *Fernando Santos Osório***Semestre:** 2026/1**Horário:** Terça 9h – 13h (Prova no Lab.)**Wiki:** SSC-600-2026(fosorio)**Web:** <http://www.icmc.usp.br/~fosorio/>**DATA DA PROVA:** 29 / 06 / 2026 - P2 FinalNRO. USP: < Colocar o seu NUSP no **programa fonte**> COMO COMENTÁRIO no CÓDIGONOME : < Colocar o seu Nome no **programa fonte**> COMO COMENTÁRIO no CÓDIGO>> COLOCAR SEU NOME E NRO. USP COMO COMENTÁRIO DO PROGRAMA FONTE ENTREGUE!**PROVA FINAL – SSC0600 - ICC 1**

**QUESTÃO UNICA – Ler arquivo dados de alunos de arquivo texto (NroUSP, Nome Completo, Média Global do aluno), exibir na tela, buscar por (parte do) nome, criar uma lista com os alunos encontrados (e atualizar a informação da média), salvar no disco e exibir na tela os dados salvos.**

**O programa deve usar os conceitos vistos em aula (structs, alocação dinâmica de memória, sub-rotinas com parâmetros por valor e por referência e arquivo texto), conforme especificado mais detalhadamente a seguir. Implemente o programa descrito abaixo de acordo com o especificado e envie no RunCodes.**

**O programa deve ser enviado para o RunCodes.ICMC da Disciplina SSC0600 (Prova P2 Final)**  
**RunCodes SSC0600 - <https://runcodes.icmc.usp.br/offerings/view/169> (SSC0600 - ICC 1 Prof. FOsorio)**

**Descrição Detalhada do Programa:**

Implemente um programa que implemente as tarefas descritas abaixo (funcionalidades), implementadas na forma de sub-rotinas modulares conforme o especificado abaixo (seguir corretamente a especificação vale nota, mas não deixe de tentar fazer o programa que realize as tarefas indicadas).

1. Ler o arquivo texto do disco “alunos.txt” contendo os dados dos alunos. Os dados dos alunos são:

**Número USP:** com até 8 dígitos (formato numérico). Por exemplo: 12345678

**Nome Completo do Aluno:** Deve ser um nome SEM ACENTOS e armazenado de forma “neutra”, ou seja, todo em maiúsculas (não devem ser armazenados caracteres “invisíveis” – não ASCII padrão. Por exemplo: Fernando Santos Osorio (digitado SEM ACENTO!) ficará armazenado como FERNANDO SANTOS OSORIO (para buscas). Elimine o ‘\n’, ‘r’ e códigos ASCII abaixo de 32.

**Média Global:** Valor numérico de ponto flutuante representando a média global do aluno.

**DADOS E EXEMPLO DE ARQUIVO TEXTO DE ENTRADA – “alunos.txt” (mesmo formato do arq. de saída)**

Campos de dados: 1 dado em cada linha. Total na primeira linha seguido de conjuntos de 3 linhas com os dados de cada aluno. O arquivo termina com a marca de fim ‘#’ que indica que não tem mais dados.

**<nro. “N” total de alunos>**

**<Nro. USP do Aluno1>**

**<Nome do Aluno1>**

**<Média Global do Aluno1>**

```
...
<Nro. USP do AlunoN>
<Nome do AlunoN>
<Média Global do AlunoN>
#
```

- O programa inicia lendo TODO o arquivo para a memória, usando uma alocação dinâmica para criar 2 estruturas de dados (crie um typedef/struct para armazenar os dados) que possam armazenar todos os dados dos alunos. Uma estrutura deve armazenar TODOS os alunos lidos e a outra vai armazenar o resultado da busca pelo nome, devendo ter o mesmo tamanho da estrutura principal (se todos alunos tiverem o mesmo nome, na busca vamos obter a lista completa “duplicada”). Sub-rotina: ler\_arquivo.

Exemplo de arquivo de entrada: “alunos.txt”

```
3
12345678
Fulano da Silva e Silva
5.00
87654321
Jose Joao da Silva
7.00
99999999
Fernando Santos Osorio
10.00
#
```

2. Exibir na tela os dados lidos dos alunos a partir do arquivo. Note que os nomes já vão estar em maiúsculas (Código ASCII do ‘a’ – 32 é ‘A’ # Código de ‘a’ é 97 e o Código de ‘A’ é 65). Exibir no mesmo formato do arquivo de entrada: total de alunos, 3 linhas com nro.USP, nome e média, até o ‘#’. Sub-rotina: exhibe\_alunos.

3. Ler do teclado (com FGETS) uma parte do nome do aluno procurado. É IMPORTANTE REMOVER os caracteres ASCII invisíveis (‘\n’, ‘\r’) e transformar em maiúsculas o texto digitado, para que possa ser encontrado este nome na busca.

4. Procurar no cadastro de alunos pelos alunos que possuem em seu nome a string de busca digitada no teclado. Podem ser encontrado mais de um aluno que possuem o mesmo nome, por exemplo, considerando o exemplo do arquivo acima, temos 2 alunos SILVA. Os alunos encontrados devem ser copiados para uma lista de alunos buscados. Esta lista tem os mesmos dados do cadastro e foi criada junto com a leitura do arquivo – alocação dinâmica de 2 estruturas, o cadastro e a lista do resultado da busca. **A média global dos alunos buscados e encontrados deverá ser alterada para 100.00.** Sub-Rotina: procura\_alunos.

ATENÇÃO: use a função do “C” **strstr** para busca de sub-strings (man 3 strstr no google!)

ATENÇÃO: as 2 strings não podem conter caracteres ASCII especiais (sem invisíveis, sem acentos) e devem estar com os caracteres em LETRAS MAIÚSCULAS para que a busca seja possível.

ATENÇÃO: alterar a média dos alunos que foram encontrados na busca para 100.00

5. Gravar no disco um arquivo texto de nome “saída.txt” com a lista dos alunos selecionados na busca. Note que a rotina que grava em disco é praticamente a mesma da exhibe\_alunos, porém o que muda é a entrada, uma vez que é passado como parâmetro a lista de alunos buscados/encontrados, em vez da lista geral. Sub-Rotina: grava\_alunos.

6. Execute o comando em “C”: **system(“cat saída.txt”);** no final de seu programa, pois ele vai permitir a exibição na tela dos dados salvos em disco, e depois termine a execução do programa. No RunCodes.ICMC em Linux é “cat saída.txt” no Windows se quiser usar esse comando é “type saída.txt”.

Se o usuário digital “Silva” na busca a saída na tela vai exibir (saída do comando *system*):

```
2
12345678
FULANO DA SILVA E SILVA
100.00
87654321
JOSE JOAO DA SILVA
100.00
#
```

#### SUB-ROTINAS A SEREM IMPLEMENTADAS:

#### DECLARAÇÃO E DOCUMENTAÇÃO DAS SUB-ROTINAS USADAS NO PROGRAMA

```
// Rotina le_arquivo: Le o arquivo texto de dados com os dados dos alunos
// Entrada: nome do arquivo de entrada
// Saída: -1 = erro, ou, nro de alunos lidos (quantidade de alunos)
// Atenção: os nomes lidos devem ser convertidos para maiúscula e só ASCII padrão.
// Atenção: a rotina le_arquivo já aloca dinamicamente os 2 vetores (cria cadastro
//          de alunos e cria lista de alunos onde ficará o resultado da busca).
int le_arquivo(char *nome);

// Rotina exhibe_alunos: Exibe os dados dos alunos armazenados no vetor criado.
// Entrada: lista é o vetor que contém os dados dos alunos e nalunos é a número
//          de alunos armazenados neste vetor.
// Exibe na tela, onde o nome está em maiúsculas e sem caracteres ASCII especiais
// e a média global do aluno é exibida com 2 casas após a vírgula.
// Exibe no mesmo formato do arquivo de entrada (qtde, dados, '#' no final).
void exhibe_alunos(t_aluno *lista, int nalunos);

// Rotina procura_alunos: recebe como parâmetros o vetor de todos alunos (cad),
// e o número de alunos cadastrados neste vetor (nalunos), assim como o vetor
// onde vai colocar os alunos encontrados na busca (lista) e quantos alunos
// foram encontrados na busca (totalalunos - parâmetro passado por referência!)
// Além disso, recebe a (sub)string de busca do nome, já em maiúsculas, que será
// usada na busca (nomparcial), e o valor da nova média a ser atualizada nos
// alunos encontrados na busca (novamedia).
void procura_alunos(t_aluno *cad, int nalunos, t_aluno *lista, int *totalalunos,
char *nomparcial, double novamedia);

// Rotina grava_alunos: grava em disco um arquivo texto "saída.txt" no mesmo
// formato do arquivo de entrada. Este arquivo deve conter a lista dos alunos
// encontrados na busca realizada (somente os alunos encontrados, com os dados
// atualizados: nomes em maiúscula e nova média global).
// Entrada: lista é o vetor que contém os dados dos alunos encontrados na busca
//          e nalunos é a número de alunos armazenados neste vetor.
void grava_alunos(char *nome, t_aluno *lista, int nalunos);
```

BOA PROVA!!!

---

## **REGRAS EM RELAÇÃO REALIZAÇÃO DESTA PROVA**

1. A PROVA É INDIVIDUAL com consulta Papel (Livros, papel impresso ou escrito) e formato Digital (Internet, Wiki, Pendrive), porém SEM CONSULTAR ou SE COMUNICAR COM HUMANOS ou QUALQUER OUTRA FORMA DE VIDA TERRESTRE ou EXTRA-TERRESTRE!  
Não podem usar de formas de comunicação com pessoas externas, além do professor, referente a prova, seja por celular (manter desligado/guardado), por e-mail, por whatsapp, por mensagens ou fóruns (“ao vivo”), com colegas, etc.  
**NÃO É PERMITIDO O EMPRÉSTIMO DE MATERIAL** (Cadernos, Anotações, Livros, etc).
2. A PROVA DEVE SER REALIZADA USANDO OS COMPUTADORES DO LABORATÓRIO.  
Não será permitido o uso de Notebooks, Tablets ou Celulares de uso pessoal.
3. A PROVA DEVE SER ENTREGUE via RunCodes.icmc (pode submeter várias vezes)  
Entrega é via <https://runcodes.icmc.usp.br/> Disciplina SSC0600 – Prova P2 FINAL
4. EM CASO DE PROBLEMAS GRAVES no RunCodes.icmc pode ser entregue por E-Mail para: [fosorio@icmc.usp.br](mailto:fosorio@icmc.usp.br) com cópia (Cc:) para [fosorio@gmail.com](mailto:fosorio@gmail.com) => **PROFESSOR DEVE SER AVISADO!**  
Com assunto/subject: **Prova-P2 <Seu\_Nome> <Nro\_USP>**  
**ANEXANDO O PROGRAMA FONTE (“.C” e o Project “.CBP”) – NÃO ANEXAR EXECUTÁVEIS!**  
**NÃO ANEXAR OBJ, BIN e EXE!!!** Seu e-mail não será recebido se for anexado bin, obj ou exe!
5. A PROVA USUALMENTE DEVE SER REALIZADA USANDO O CODEBLOCKS DO LABORATÓRIO, e enviada no RUNCODES, se for usar outro Compilador/IDE, INDIQUE NO E-MAIL qual AMBIENTE, COMPILADOR e IDE usou!
6. **AO ENTREGAR A PROVA NO RUNCODES ou POR E-MAIL O(A) ALUNO(A) CONCORDA COM ESTAS REGRAS E SE COMPROMETE A FAZER A PROVA INDIVIDUALMENTE!**

---

FIM